

CODE SPORT



In this month's column, we feature a list of interview questions on operating systems, algorithms and computer architecture.

Over the past few months, we have been discussing natural language processing and insight-centric storage systems. This month, we take a break from that discussion to explore a bunch of computer science interview questions. Let us start off on a light-hearted note.

1. What is the KISS Principle in computer science?
2. File systems and database systems both act as containers of user data, albeit in different ways. Can you differentiate the two in terms of the consistency guarantees they provide to the end user?
3. Consider a file '1.txt' on the same file system, say ext4, on Linux, which is opened by two different processes. Both these processes modify overlapping regions of the file and close their individual file handles. While each process has its own file handle on which it operates, how many file descriptors are actually created for this file in the file system in the kernel? If both process1 and process2 write 'a' and 'b' as the value at logical offset '0' of the file '1.txt', what would actually be stored when these processes close their descriptive handles?
4. When the user invokes 'ls' on the root directory of your file system, can you explain what Linux system calls get invoked internally?
5. What is a FUSE file system? How is it different from a kernel level file system module? Can you reimplement the Linux ext4 file system as a FUSE module? If not, why not?
6. Instead of having local storage on a hard disk connected to the device, you are asked to design a file system which will store its data on a public cloud storage provider like Amazon S3, which has an object interface. The file system needs to provide normal file access semantics to local applications, but has to store its durable data in the cloud. What are the design considerations when

creating such a system?

7. We all know that databases support ACID semantics where the 'A' in ACID stands for atomicity. Atomicity means that either the transaction completes successfully in its entirety or it appears not to have started at all. For example, consider the simple example of transferring money from your account to your father's account. Successful completion means that the money is debited from your account and credited to your father's account. So if there is a failure while the transaction is in progress, say after the money has been debited from your account but before it is credited to your father's account, then the transaction is aborted and the effect to the end user is as if the transaction never started at all. This means that it appears as if the money was never debited from your account. Can you explain the mechanism by which databases support such failure atomicity? Think of how it would be possible to roll back a partially completed transaction.
8. What is a gossip protocol? How would you implement it in a local network of computers?
9. The conventional wisdom with regard to traditional storage media is that interrupt based signalling of IO completion is more efficient than polling. Can you think of a situation where polling may be better than interrupt based IO completion signalling? There is an interesting research paper which discusses this issue in detail, available at https://www.usenix.org/legacy/events/fast/tech/full_papers/Yang.pdf
10. What are content addressable memories as opposed to traditional random access memories?
11. You are asked to write a program which supports the following operations: